

Sistema de Gestão de Encomendas: Análise, Modelação e Conceção de uma Solução Orientada a Objetos

David Soares¹, Mário Figueira¹ e Masembo Zola¹

¹ Instituto Superior de Engenharia do Porto, Departamento de Engenharia Eletrotécnica,
Rua Dr. António Bernardino de Almeida 431, 4249-015 Porto, Portugal
{1251971, 1251824, 1251825}@isep.ipp.pt

Resumo. Este relatório descreve a análise e conceção de um sistema de gestão de encomendas orientado a objetos, a implementar em C++, que servirá de apoio ao transporte de encomendas entre seis pontos de distribuição numa pequena cidade. O trabalho cobre a engenharia de requisitos, a modelação conceptual do domínio, a especificação de user stories e casos de uso, os diagramas de sequência de sistema e o desenho interno da aplicação, incluindo diagramas de classes, de estados, de pacotes, de componentes e de instalação. Aplicam-se notação UML e princípios GRASP para garantir elevada coesão e baixo acoplamento. O sistema preconizado visa aumentar a eficiência operacional, reduzir falhas no processo logístico e proporcionar ao utilizador informação atualizada sobre o estado das suas encomendas. Encontra-se ainda em fase de planeamento, pelo que este documento descreve o sistema tal como está previsto.

Palavras-chave: Gestão de Encomendas · Engenharia de Software · C++ · UML · GRASP · Programação Orientada a Objetos · Sistemas de Informação.

1 Introdução

O envio de encomendas entre pessoas é uma necessidade do dia-a-dia, seja para fazer chegar um presente a um familiar, despachar documentos entre empresas ou simplesmente trocar objetos entre vizinhos. Em contextos urbanos de grande dimensão esta atividade encontra-se bem servida por operadores logísticos consolidados, mas em contextos locais — como uma pequena cidade com apenas alguns pontos de distribuição — a coordenação entre quem envia, quem recebe e os meios de transporte disponíveis é frequentemente feita de forma improvisada, recorrendo a telefonemas, registos em papel ou folhas de cálculo partilhadas. O sistema encontra-se atualmente em fase de implementação, tendo já sido desenvolvidas as principais funcionalidades definidas na fase de análise e desenho.

1.2 Problema

A ausência de um sistema estruturado tem consequências práticas relevantes. Por um lado, torna-se difícil saber, num determinado momento, que veículos estão disponíveis, que encomendas aguardam recolha e qual o estado de cada entrega. Por outro lado, o risco de erro humano aumenta com o volume de operações, o que pode conduzir a encomendas extraviadas, atrasos injustificados ou atribuições de veículos inadequadas ao peso e ao volume do que se pretende transportar. Este problema torna-se ainda mais evidente quando

existem múltiplos pontos de recolha e entrega e quando os recursos de transporte são limitados.

1.3 Abordagem Proposta

Propõe-se o desenvolvimento de um sistema que permita simplificar e otimizar a gestão de encomendas numa pequena cidade com seis pontos de distribuição. Pretende-se que o utilizador possa descrever a sua encomenda, indicar o ponto de origem e o ponto de destino, e que o próprio sistema se encarregue de escolher, de entre os veículos disponíveis, aquele que deverá efetuar o transporte. Em paralelo, o sistema deverá acompanhar a encomenda ao longo do processo, mantendo o estado atualizado desde a criação do pedido até à entrega final. A solução é desenhada seguindo boas práticas de engenharia de software [2, 3], recorrendo a notação UML [4] e aos padrões de atribuição de responsabilidades GRASP [1].

Importa sublinhar, desde já, que o sistema aqui descrito se encontra ainda em fase de análise e conceção: a implementação propriamente dita, em linguagem C++, será realizada numa fase posterior. Assim, tudo o que se segue deverá ser interpretado como uma proposta detalhada daquilo que o sistema fará quando for construído, e não como uma descrição de software já em funcionamento.

Para tornar este documento acessível tanto a leitores com formação em programação como a leitores sem esse tipo de conhecimentos, ao longo do texto são introduzidos e explicados, em linguagem simples, os principais conceitos técnicos utilizados — nomeadamente user story, caso de uso, diagrama de sequência, modelo de domínio, padrões GRASP e requisitos FURPS+.

1.4 Estrutura do Documento

O presente documento encontra-se organizado em onze capítulos. No Capítulo 2 são apresentados os objetivos gerais e específicos do projeto. O Capítulo 3 descreve as user stories que capturam as necessidades dos utilizadores. No Capítulo 4 é feita a engenharia de requisitos, abrangendo requisitos funcionais, não funcionais (segundo o modelo FURPS+), considerações sobre persistência de dados e a matriz de rastreabilidade entre requisitos e casos de uso. O Capítulo 5 apresenta o modelo de domínio, e o Capítulo 6 o modelo de casos de uso, incluindo a especificação detalhada de quatro casos de uso representativos. O Capítulo 7 expõe os diagramas de sequência de sistema correspondentes a esses casos de uso. O Capítulo 8 é dedicado ao desenho do sistema, abordando diagrama de classes, diagrama de sequência, diagrama de estados da Encomenda, aplicação de padrões GRASP, e diagramas de pacotes, componentes e instalação. O Capítulo 9 apresenta o glossário de termos, o Capítulo 10 contém as conclusões e, finalmente, o Capítulo 11 lista as referências bibliográficas utilizadas.

2 Objetivos do Projeto

O objetivo geral deste projeto consiste em desenvolver um sistema orientado a objetos para a gestão de encomendas, a ser implementado em C++, que permita registar clientes, criar e acompanhar encomendas e atribuir automaticamente veículos disponíveis para o respetivo transporte, tudo isto com base na aplicação sistemática de princípios de Engenharia de Software [2, 6].

Para concretizar este objetivo geral, foram definidos objetivos específicos que orientam o trabalho de análise e concepção. Pretende-se, em primeiro lugar, que o sistema permita o registo de clientes, mantendo a informação necessária para a comunicação e para o acompanhamento dos seus pedidos. Pretende-se igualmente que seja possível criar encomendas, indicando o depósito de origem, o depósito de destino e os dados de transporte relevantes, nomeadamente o peso e uma breve descrição. Um terceiro objetivo passa pela gestão da frota de veículos, incluindo o registo de novos veículos, a consulta da sua disponibilidade e a sua eventual remoção quando se encontrem avariados ou indisponíveis.

A atribuição de veículos às encomendas deverá ser efetuada pelo próprio sistema, de forma automática, tendo em conta a disponibilidade e a capacidade de carga. Paralelamente, deverá ser possível acompanhar, em qualquer momento, o estado de uma encomenda, desde a sua criação até à entrega.

Por fim, e de forma transversal, o projeto tem como objetivo pedagógico a aplicação de notação UML [4] e dos princípios GRASP [1] na estruturação da solução, garantindo assim boas práticas de modelação e de atribuição de responsabilidades entre classes.

3 User Stories

Uma user story é uma forma curta e informal de descrever uma funcionalidade do sistema na perspetiva de quem a vai utilizar. Normalmente segue a estrutura "Como [tipo de utilizador], quero [fazer algo], para que [obter um benefício]". Esta formulação, simples de compreender mesmo por quem não tem formação em programação, permite focar a equipa de desenvolvimento naquilo que realmente importa: o valor que cada funcionalidade traz ao utilizador final.

No contexto deste projeto foram identificados dois tipos de utilizadores: o cliente, que recorre ao sistema para enviar e acompanhar encomendas, e o administrador, responsável pela supervisão global da plataforma e pela gestão dos recursos logísticos, nomeadamente veículos e contas de clientes. A cada user story foi atribuído um identificador (US01 a US09) que facilita a sua referência em fases posteriores do projeto, nomeadamente na matriz de rastreabilidade apresentada na secção 4.4.

Na perspetiva do cliente. São consideradas as user stories US01 a US04. Em US01, enquanto cliente, pretende-se poder criar uma encomenda, de modo a enviar um objeto de um depósito para outro. Em US02, para que esse processo funcione corretamente, o cliente deverá poder selecionar o depósito de origem e o depósito de destino, garantindo assim que a recolha e a entrega são feitas nos pontos certos. Em US03, uma vez criada a encomenda, o cliente deverá poder consultar o seu estado a qualquer momento, de modo a saber em que fase do processo de entrega ela se encontra. Em US04, o cliente deverá poder obter uma estimativa razoável sobre a evolução da entrega, o que lhe permitirá ter previsibilidade sobre o transporte.

Na perspetiva do administrador. São consideradas as user stories US05 a US09. Em US05, enquanto administrador, pretende-se poder adicionar veículos ao sistema, para assegurar que existem meios de transporte disponíveis para realizar as encomendas. Em US06, deverá ser possível remover veículos quando estes estiverem indisponíveis ou avariados. Em US07, o administrador deverá poder remover contas de clientes, garantindo a integridade e a segurança dos dados. Em US08, deverá ser possível consultar informações globais sobre clientes, encomendas, veículos e depósitos, de forma a supervisionar o funcionamento geral da plataforma. Por último, em US09, deverá ser possível saber, para

cada encomenda, qual o veículo que a transporta e qual o depósito de destino, assegurando um controlo efetivo sobre o processo logístico.

4 Engenharia de Requisitos

A engenharia de requisitos é a disciplina que se ocupa de identificar, documentar e validar aquilo que um sistema deverá fazer e em que condições o deverá fazer [2, 5]. Distinguem-se, habitualmente, dois grandes grupos de requisitos: os requisitos funcionais, que descrevem as funcionalidades concretas que o sistema deve oferecer, e os requisitos não funcionais, que especificam restrições de qualidade, como a rapidez, a fiabilidade ou a facilidade de utilização.

4.1 Requisitos Funcionais

Em termos funcionais, o sistema deverá permitir registar clientes, mantendo a informação mínima necessária para os identificar e contactar. Deverá também permitir criar encomendas, associando-as a um cliente e a dois depósitos — o de origem e o de destino. Será necessário listar os veículos disponíveis a qualquer momento, de modo que o sistema possa selecionar automaticamente o veículo adequado para cada encomenda, e atualizar o estado das entregas à medida que estas progridem, desde o momento em que a encomenda é registada até à sua entrega no destino. Adicionalmente, o sistema deverá permitir ao administrador adicionar e remover veículos da frota, remover contas de clientes e consultar informação agregada sobre todos os recursos geridos pela aplicação.

4.2 Requisitos Não Funcionais — Modelo FURPS+

O modelo FURPS+ [7] é uma grelha de classificação de requisitos não funcionais que organiza as qualidades exigidas a um sistema em cinco categorias principais — Functionality, Usability, Reliability, Performance e Supportability — às quais se acrescenta um grupo "+" onde se colocam restrições adicionais, como as tecnológicas ou legais. A sigla é útil precisamente por ajudar a equipa de desenvolvimento a não se esquecer de nenhuma dimensão relevante.

No que respeita à usabilidade (Usability), pretende-se que o sistema apresente uma interface simples em modo de consola, adequada ao contexto académico do projeto e de utilização intuitiva mesmo para quem não está habituado a interagir com software técnico. Em termos de fiabilidade (Reliability), o sistema deverá garantir que os estados das encomendas são consistentes ao longo de todo o ciclo de entrega, evitando, por exemplo, que uma mesma encomenda surja simultaneamente como pendente e como entregue — consistência essa assegurada, a nível de desenho, pelo diagrama de estados apresentado na secção 8.3. Quanto ao desempenho (Performance), espera-se uma resposta imediata às ações do utilizador, uma vez que as operações previstas são de pequena dimensão e executadas localmente. Em matéria de suportabilidade (Supportability), o código deverá ser legível e testável, de modo a permitir a sua evolução futura. Por último, no grupo das restrições (Constraints), o projeto impõe o uso da linguagem C++ e a execução local da aplicação.

4.3 Persistência de Dados

Atendendo à natureza académica e local do projeto, considera-se que, numa primeira versão, os dados do sistema — clientes, encomendas, veículos e depósitos — serão mantidos apenas em memória, durante a execução da aplicação. Todos os dados introduzidos serão, portanto, perdidos quando a aplicação terminar. Esta decisão simplifica a implementação inicial e permite focar o esforço de desenvolvimento na correta modelação do domínio e na aplicação dos princípios de engenharia de software que constituem o objetivo pedagógico da unidade curricular.

Numa fase posterior, como linha de evolução natural do trabalho, prevê-se a introdução de persistência em ficheiros de texto (por exemplo, em formato CSV) ou em ficheiros binários, recorrendo às bibliotecas padrão de C++ para leitura e escrita em disco.

A estrutura do sistema — com clara separação entre a interface de utilizador, a camada de controlo e as classes de domínio (ver secção 8.5) — facilita essa evolução, uma vez que a persistência poderá ser adicionada através de classes auxiliares sem alterar a lógica central da aplicação.

4.4 Matriz de Rastreabilidade

A matriz de rastreabilidade (Tabela 1) estabelece a correspondência entre cada user story e o caso de uso, ou conjunto de casos de uso, que a realizam. Este tipo de matriz é particularmente útil para garantir que todas as necessidades identificadas na fase de levantamento de requisitos têm efetivamente uma funcionalidade associada no desenho do sistema, e, simetricamente, que todos os casos de uso especificados têm uma justificação clara na perspetiva do utilizador.

Tabela 1 –Matriz de rastreabilidade entre user stories e casos de uso.

US	Descrição resumida	Caso(s) de Uso
US01	Criar uma encomenda.	UC1 —CriarEncomenda
US02	Selecionar depósitos de origem e destino.	UC1 (incluído)
US03	Consultar o estado de uma encomenda.	UC2 — Consultar Estado
US04	Obter estimativa sobre a evolução da entrega.	UC2 (estendido)
US05	Adicionar veículos ao sistema.	UC4 — Adicionar Veículo
US06	Remover veículos do sistema.	UC — Remover Veículo
US07	Remover contas de clientes.	UC — Remover Cliente

US08	Consultar informação global.	UC3 — Consultar Info Global
US09	Saber qual o veículo e destino de cada encomenda.	UC3 (detalhe)

5 Modelo de Domínio

O modelo de domínio (Domain Model) é uma representação visual dos principais conceitos do mundo real que o sistema pretende modelar, juntamente com as relações estáticas entre eles [1]. Não se trata de um diagrama técnico sobre como o software vai ser construído, mas sim de uma forma de capturar, numa só imagem, o vocabulário do negócio — neste caso, o vocabulário da logística de encomendas. É por isso uma ferramenta particularmente útil para alinhar a equipa de desenvolvimento com os utilizadores e com eventuais clientes do sistema, ainda antes de se começar a programar.

No contexto deste projeto, o modelo de domínio reflete a realidade de uma pequena cidade com seis pontos de distribuição, onde clientes enviam encomendas entre si e veículos fazem o transporte entre esses pontos. Pretende-se, com este modelo, fornecer uma visão conceptual do problema, identificando as entidades fundamentais envolvidas no envio, transporte e entrega de encomendas, sem qualquer referência a pormenores de implementação.

5.1 Conceitos de Domínio

Foram identificados quatro conceitos principais: Cliente, Encomenda, Veículo e Depósito. O Cliente representa o utilizador do sistema responsável pela criação de encomendas. A Encomenda corresponde ao pedido de transporte efetuado por um cliente, contendo as informações essenciais para que o processo de entrega possa ser conduzido e acompanhado. O Veículo representa o meio de transporte utilizado para deslocar as encomendas entre os diferentes pontos da cidade. Finalmente, o Depósito corresponde a um ponto físico de recolha ou entrega, existindo vários desses pontos distribuídos pela área de operação do sistema.

5.2 Relações e Associações

Entre estes conceitos estabelecem-se várias associações. Cada Cliente pode criar encomendas no sistema, existindo por isso uma associação direta entre Cliente e Encomenda. Cada Encomenda está associada a dois Depósitos distintos — o de origem e o de destino — o que permite modelar explicitamente os pontos físicos envolvidos no processo de entrega. Adicionalmente, cada Veículo encontra-se associado às encomendas que transporta, sendo responsável pela deslocação destas entre o depósito de origem e o depósito de destino. No seu conjunto, estas associações permitem representar de forma clara o fluxo logístico do sistema.

5.3 Multiplicidades

No que se refere às multiplicidades, cada Cliente pode criar zero ou mais Encomendas, ao passo que cada Encomenda está associada a exatamente um cliente. Cada encomenda possui obrigatoriamente um Depósito de origem e um Depósito de destino, garantindo que os locais de recolha e de entrega estão sempre definidos. Um Veículo pode transportar várias encomendas ao longo do tempo, mas cada encomenda é transportada por um único veículo durante o seu ciclo de entrega.

5.4 Atributos

Cada entidade do modelo de domínio é caracterizada por um conjunto de atributos relevantes. O Cliente possui um identificador único, um nome e um contacto. A Encomenda é caracterizada por um identificador, uma descrição, um peso, um estado, um identificador do cliente, um identificador do depósito de origem, um identificador do depósito final e um do seu respetivo veículo, permitindo acompanhar o seu progresso ao longo do processo de entrega. O Veículo inclui atributos como um identificador, matrícula, capacidade máxima e estado de disponibilidade, ao passo que o Depósito é descrito por um identificador, um nome, uma localização e uma capacidade máxima de armazenamento.

5.5 Diagrama do Modelo de Domínio

A Figura 1 apresenta o diagrama do modelo de domínio, sintetizando graficamente os conceitos, os atributos, as associações e as multiplicidades descritos nas subsecções anteriores.

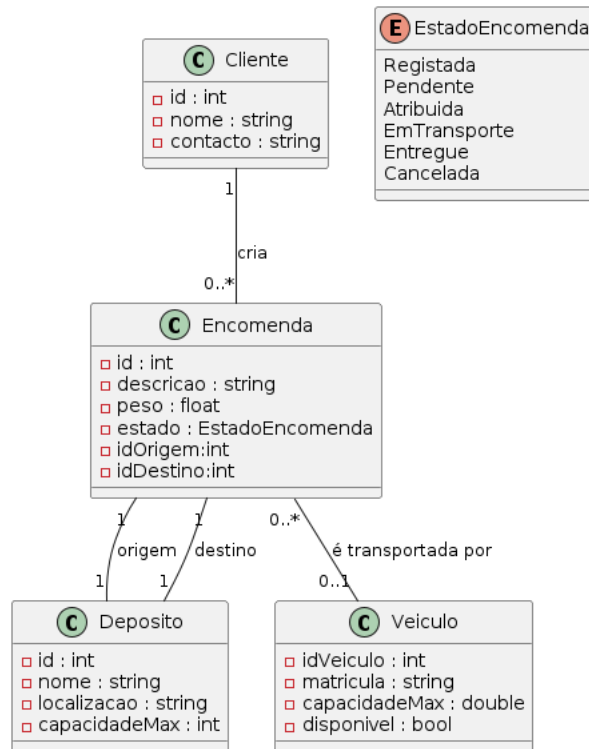


Figura 1 - Modelo de domínio do sistema de gestão de encomendas, evidenciando as entidades Cliente, Encomenda, Veículo e Depósito e as respetivas associações e multiplicidades.

6 Modelo de Casos de Uso

Um caso de uso (use case) descreve uma forma concreta como um utilizador — designado por ator — interage com o sistema para atingir um determinado objetivo [1, 4]. Ao contrário do modelo de domínio, que se foca nos conceitos, o modelo de casos de uso foca-se no comportamento do sistema na perspetiva do utilizador. Cada caso de uso representa assim um "pedaço" de funcionalidade visível do exterior, pelo que é uma ferramenta fundamental para garantir que o sistema entrega, efetivamente, aquilo que os seus utilizadores dele esperam.

No presente projeto foram identificados dois atores externos: o Cliente, que é o ator principal responsável pela criação e acompanhamento de encomendas, e o Administrador, ator secundário responsável pela supervisão geral da plataforma e pela gestão de veículos e de contas. Para além destes, o próprio Sistema desempenha, internamente, um papel ativo, nomeadamente ao atribuir automaticamente um veículo a cada nova encomenda.

6.1 Diagrama de Casos de Uso

A **Figura 2** ilustra as funcionalidades disponibilizadas pelo sistema a cada um dos atores, assim como as relações de inclusão entre casos de uso. O caso de uso "Criar Encomenda",

por exemplo, inclui os casos de uso "Selecionar Depósito de Origem", "Selecionar Depósito de Destino" e "Atribuir Veículo à Encomenda", sinalizando que estes passos fazem parte integrante do fluxo de criação de uma encomenda.

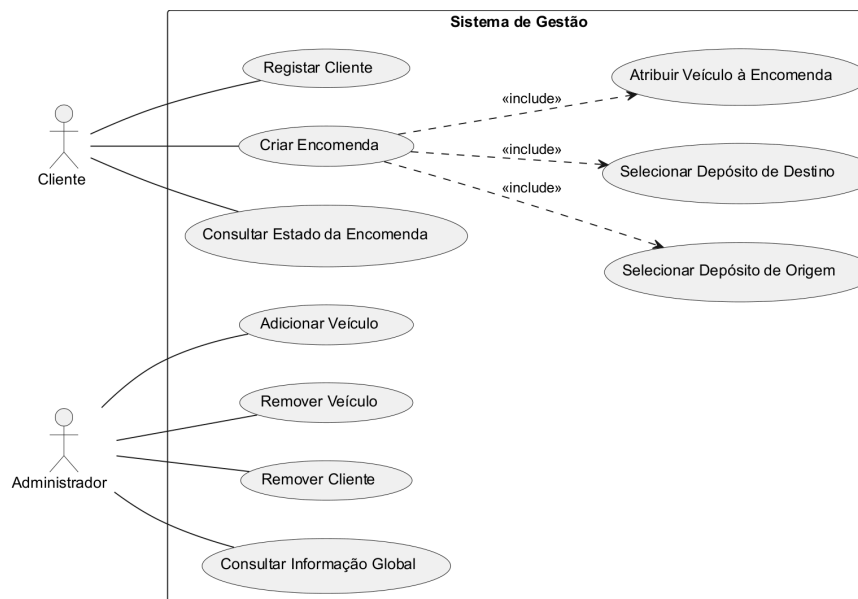


Figura 2 - Diagrama de casos de uso do sistema de gestão de encomendas, com os atores Cliente e Administrador e as funcionalidades que cada um pode invocar.

6.2 Especificação dos Casos de Uso

De seguida apresentam-se, em detalhe, quatro casos de uso considerados representativos do funcionamento do sistema: Criar Encomenda e Consultar Estado da Encomenda, do lado do Cliente, e Consultar Informação Global e Adicionar Veículo, do lado do Administrador. Em cada caso são indicados o ator principal, a descrição, as pré-condições, as pós-condições, o fluxo principal, os fluxos alternativos e as regras de negócio aplicáveis.

6.2.1 UC 1 — Registrar Cliente

A Tabela 2, mostra as especificações do caso de uso registrar cliente.

Tabela 2. Especificação do caso de uso registrar cliente

Ator Principal	Cliente
Nome	Registrar Cliente
Descrição	É o primeiro passo do sistema, permitindo a criação do usuário.

Pré-condições	O sistema é inclusivo para todos, sem registações de idade
Pós-condições	Após o registo, o cliente é armazenado no sistema e recebe um número de identificação
Fluxo Principal	1. Utilizador faz cadastro 2. Sistema cria o cliente 3. Sistema gera automaticamente o número de identificação (ID) 4. Cliente fica registado 5. Sistema devolve o número de identificação (ID)
Fluxos Alternativos	2a. Nome inválido 2a1. Sistema rejeita o registo 2a2. Sistema informa erro 2b. Contacto inválido 2b1. Sistema rejeita registo 2c. Contacto Já existe 2c1. Sistema rejeita registo
Regras de Negócios	Só é possível fazer o cadastro uma vez.

6.2.2 UC 2 — Criar Encomenda

A Tabela 3, mostra as especificações do caso de uso Criar Encomenda.

Tabela 3. Especificação do caso de uso criar encomenda.

Ator Principal	Cliente
Nome	Criar Encomenda
Descrição	Permite ao cliente criar uma nova encomenda no sistema, definindo o depósito de origem e o depósito de destino do transporte.
Pré-condições	O cliente deverá encontrar-se registado no sistema. Deverão existir, pelo menos, dois depósitos disponíveis.
Pós-condições	A encomenda fica registada no sistema.

	Um veículo disponível é associado automaticamente à encomenda. O estado inicial da encomenda é definido.
Fluxo Principal	1. O cliente inicia o processo de criação de encomenda. 2. O sistema solicita os dados da encomenda. 3. O cliente seleciona o depósito de origem. 4. O cliente seleciona o depósito de destino. 5. O sistema valida e regista a encomenda. 6. O sistema atribui automaticamente um veículo disponível.
Fluxos Alternativos	5a. Dados da encomenda inválidos ou incompletos: o sistema solicita a correção das informações antes de prosseguir. 6a. Não existem veículos disponíveis: o sistema regista a encomenda com estado pendente e informa o cliente dessa situação.
Regras de Negócio	O depósito de origem e o depósito de destino têm de ser distintos. O peso da encomenda tem de ser superior a zero.

6.2.3 UC 3 — Consultar Estado da Encomenda

A tabela 4, mostra as especificações do caso de uso Consultar Estado da Encomenda.

Tabela 4. Especificação do caso de uso consultar estado da encomenda

Ator Principal	Cliente
Nome	Consultar Estado da Encomenda
Descrição	Permite ao cliente acompanhar o estado atual de uma encomenda sua, desde a recolha no depósito de origem até à entrega no destino.
Pré-condições	O cliente deverá encontrar-se registado no sistema. A encomenda a consultar deverá estar associada a esse cliente.
Pós-condições	O cliente obtém a informação atualizada sobre o estado da encomenda.
Fluxo Principal	1. O cliente solicita a consulta do estado de uma encomenda. 2. O sistema solicita a identificação da encomenda. 3. O cliente fornece o identificador da encomenda.

	4. O sistema apresenta o estado atual da encomenda.
Fluxos Alternativos	3a. Identificador inválido ou inexistente: o sistema informa o cliente e solicita nova introdução.
Regras de Negócio	Só é possível consultar encomendas associadas ao próprio cliente.

6.2.4 UC 4 — Adicionar Veículo

A Tabela 5, mostra as especificações do caso de uso adicionar veículos.

Tabela 5. Especificação caso de uso adicionar veículos

Ator Principal	Administrador
Nome	Adicionar Veículo
Descrição	Permite ao administrador adicionar um novo veículo ao sistema, definindo as suas principais características. Após o registo, o veículo fica disponível para o transporte de encomendas.
Pré-condições	O administrador deverá estar autenticado no sistema.
Pós-condições	O veículo fica registado no sistema. O veículo fica disponível para ser atribuído a encomendas futuras.
Fluxo Principal	1. O administrador seleciona a opção "Adicionar Veículo". 2. O sistema solicita os dados do veículo. 3. O administrador introduz a matrícula, a capacidade máxima e os restantes atributos. 4. O sistema valida os dados introduzidos. 5. O sistema regista o veículo. 6. O sistema define o estado inicial do veículo como "disponível". 7. O sistema confirma ao administrador a adição do veículo.
Fluxos Alternativos	4a. Dados inválidos ou incompletos: o sistema solicita a correção antes de prosseguir com o registo.
Regras de Negócio	A matrícula do veículo tem de ser única no sistema. A capacidade máxima tem de ser superior a zero.

6.2.5 UC 5 — Remover Veículos

A Tabela 6, mostra as especificações do caso de uso remover veículos.

Tabela 6. especificações do caso de uso remover veículos

Ator principal	Administrador
Nome	Remover veículos
Descrição	Permite ao administrador remover um veículo da frota do sistema.
Pré-condições	O administrador deve estar autenticado no sistema. O veículo deve existir.
Pós-condições	O veículo é removido da frota. O veículo deixa de estar disponível no sistema.
Fluxo Principal	1.O administrador seleciona a opção Remover Veículo. 2.O sistema solicita o ID do veículo. 3.O administrador introduz o ID. 4.O sistema procura o veículo. 5.O sistema remove o veículo. 6.O sistema informa que o veículo foi removido com sucesso
Fluxo alternativo	4a. Veículo inexistente 4a1. O sistema não encontra o veículo. 4a2. O sistema informa que o veículo não existe. 5a. Veículo associado a entrega ativa 5a1. O sistema detecta que o veículo está associado a uma encomenda em transporte.

	5a2. O sistema impede a remoção do veículo.
Regras de Negócio	Cada veículo possui um ID único. Apenas veículos registrados podem ser removidos. Um veículo associado a uma entrega ativa não pode ser removido. Apenas o administrador pode remover veículos

6.2.6 UC 6 — Remover Cliente

A Tabela 7, mostra as especificações do caso de uso remover cliente.

Tabela 7.Especificação do caso de uso remover clientes

Ator Principal	Administrador
Nome	Remover Cliente
Descrição	Permite ao administrador remover um cliente registrado do sistema.
Pré-condições	O administrador deve estar autenticado no sistema O cliente deve existir.
Pós-Condições	O cliente é removido do sistema. O cliente deixa de constar na lista de clientes registrados.
Fluxo Principal	O administrador seleciona a opção Remover Cliente. O sistema solicita o ID do cliente. O administrador introduz o ID. O sistema procura o cliente. O sistema remove o cliente. O sistema informa que o cliente foi removido com sucesso

Fluxo Alternativo	4a. Cliente inexistente 4a1. O sistema não encontra o cliente. 4a2. O sistema informa que o cliente não existe.
Regra de Negócio	Cada cliente possui um ID único. Apenas clientes registrados podem ser removidos. Apenas o administrador pode remover clientes.

6.2.7 UC 7 — Consultar Informação Global

Na Tabela 8, temos as especificações do caso de uso consultar informação geral

Tabela 8. Especificações do caso de uso consultar informação geral

Ator Principal	Administrador
Nome	Consultar Informação Global
Descrição	Permite ao administrador consultar, de forma agregada, toda a informação do sistema — clientes, encomendas, veículos e depósitos — de modo a supervisionar o seu funcionamento.
Pré-condições	O administrador deverá estar autenticado no sistema.
Pós-condições	Nenhuma alteração aos dados do sistema.
Fluxo Principal	1. O administrador solicita a consulta global. 2. O sistema apresenta a lista de clientes registrados. 3. O sistema apresenta a lista de encomendas, com os veículos atribuídos e os depósitos associados. 4. O sistema apresenta a lista de veículos e o seu estado atual. 5. O sistema apresenta a lista de depósitos e a respetiva localização.
Fluxos Alternativos	2a. Não existem registos a apresentar: o sistema informa o administrador dessa situação.

Regras de Negócio	A consulta é estritamente de leitura, não podendo originar modificações nos dados.
--------------------------	--

7 Diagramas de Sequência de Sistema

Um diagrama de sequência de sistema (System Sequence Diagram, ou SSD) é um tipo específico de diagrama UML [4] que mostra, ao longo do tempo, as mensagens trocadas entre um ator externo e o sistema visto como uma "caixa preta" [1]. O objetivo não é detalhar o funcionamento interno do software, mas sim evidenciar, de forma clara e ordenada, as interações que ocorrem em cada cenário: quem pede o quê, em que momento, e que resposta obtém do sistema. Os SSDs são, por isso, uma ponte natural entre os casos de uso — que descrevem o que acontece em texto corrido — e o desenho interno do sistema, que virá mais à frente.

Nesta secção apresentam-se os SSDs correspondentes aos sete casos de uso anteriormente detalhados, ilustrando a sequência de interações previstas entre cada ator e o sistema.

7.1 SSD: UC 1 — Registar Cliente

A Fig. 3 representa a interação entre o cliente e o sistema durante o processo de registo de um novo cliente. O utilizador inicia o processo através da interface principal, introduzindo os dados necessários, como nome e contacto. Em seguida, o sistema valida as informações fornecidas e verifica se o contacto já se encontra registado. Caso os dados sejam válidos e únicos, o cliente é criado e armazenado no repositório, sendo apresentada

uma mensagem de confirmação. Caso contrário, o sistema informa que os dados são inválidos ou que o contacto já existe.

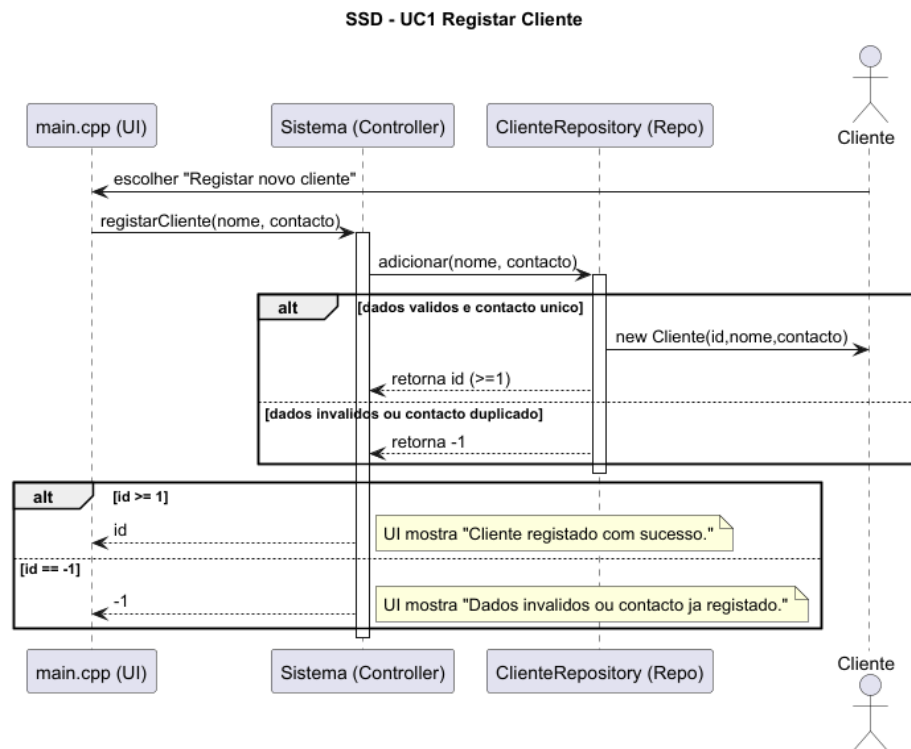


Fig. 3. SSD correspondente ao caso de uso registrar cliente.

7.2 SSD UC 2 — Criar Encomenda

A Fig. 4 representa a interação entre o cliente e o sistema durante o processo de criação de uma encomenda. O cliente inicia o processo, fornece os dados necessários e seleciona os depósitos de origem e de destino, sendo posteriormente notificado da confirmação da encomenda.

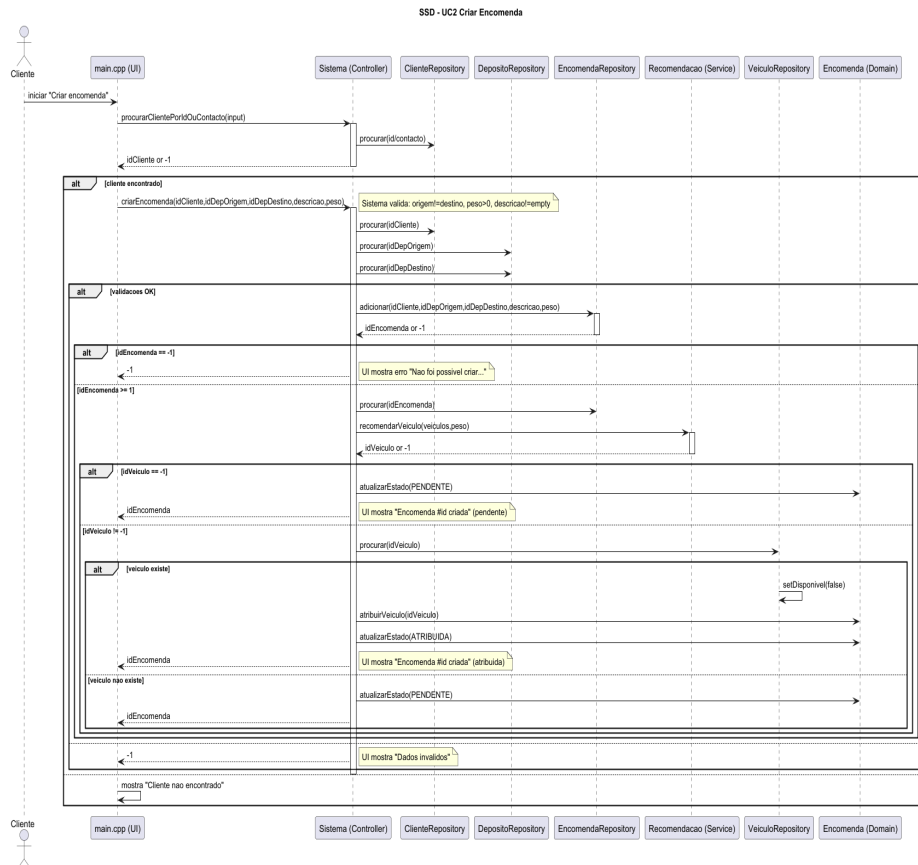


Fig. 4. SSD correspondente ao caso de uso Criar Encomenda.

7.3 SSD: UC3 — Consultar Estado da Encomenda

A Fig. 5 ilustra a sequência de eventos envolvidos na consulta do estado de uma encomenda. O cliente fornece a identificação da encomenda e o sistema devolve a informação atualizada sobre o estado em que esta se encontra.

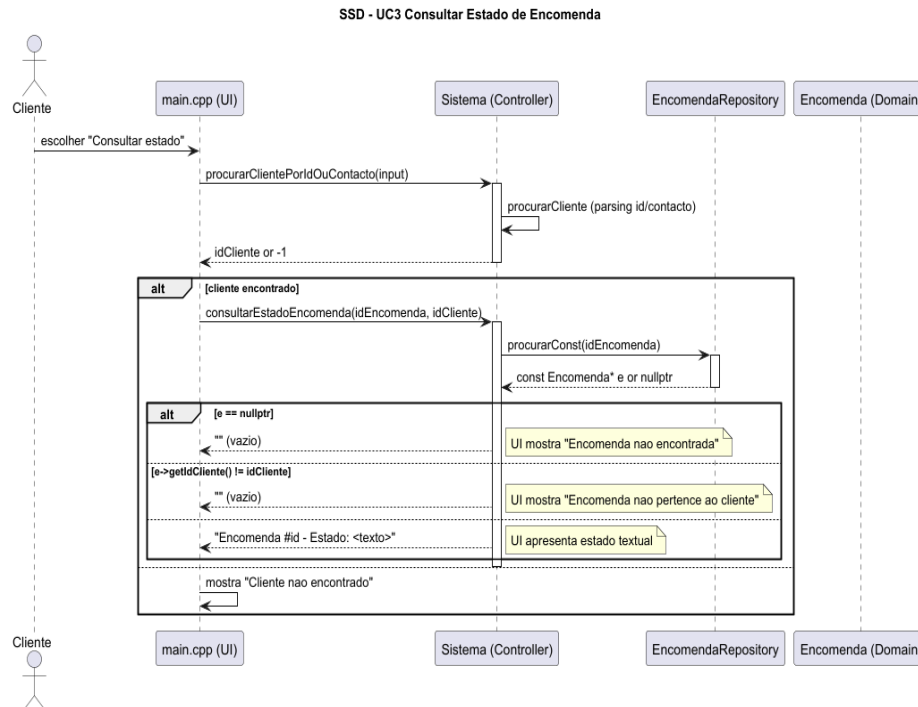


Fig. 5. SSD correspondente ao caso de uso Consultar Estado da Encomenda.

7.4 SSD: UC 4 — Adicionar Veículo

A Fig. 6 O System Sequence Diagram do caso de uso “Adicionar Veículo” ilustra o processo de registo de um novo veículo por parte do administrador. O administrador introduz os dados do veículo, como matrícula e capacidade máxima, sendo estes validados pelo sistema. O sistema verifica ainda a existência de veículos com matrícula duplicada, garantindo a unicidade dos registos. Em caso de sucesso, o veículo é registado e disponibilizado para utilização; caso contrário, o sistema informa o administrador sobre a impossibilidade de realizar a operação.

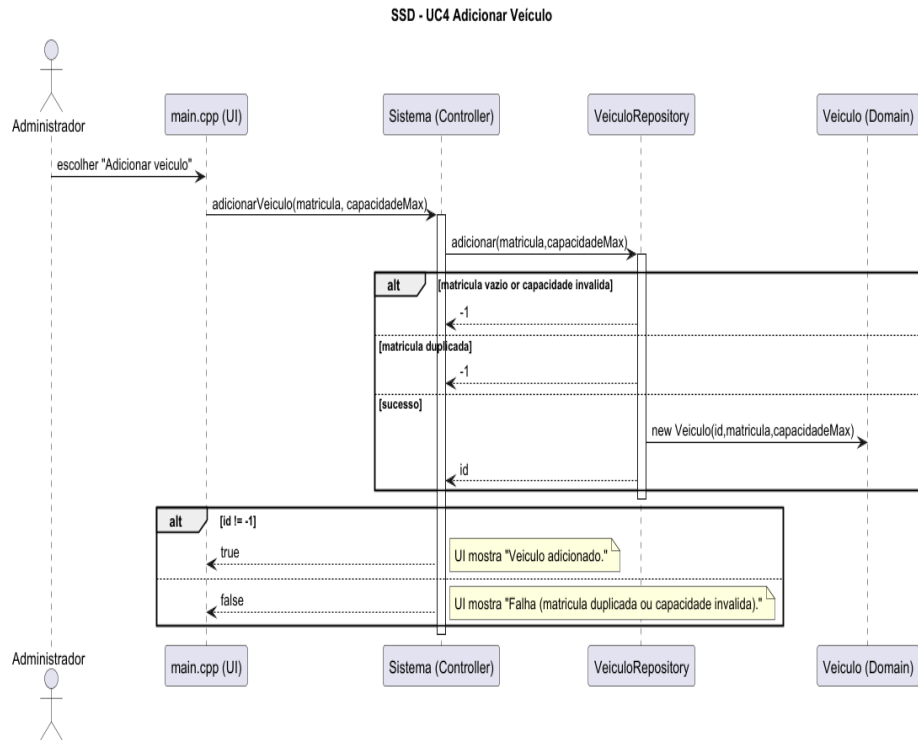


Fig. 6. SSD correspondente ao caso de uso Adicionar Veículo.

7.5 SSD: UC 5 — Remover Veículo

O System Sequence Diagram (Fig.7) associado ao caso de uso “Remover Veículo” descreve a sequência de interações necessárias para a remoção de um veículo do sistema. O administrador solicita a remoção de um veículo identificado pelo seu identificador, sendo esta operação validada pelo sistema. O sistema verifica se o veículo se encontra associado a encomendas ativas, impedindo a sua remoção em caso afirmativo. Caso não

existam restrições, o veículo é removido com sucesso e o sistema informa o administrador do resultado da operação.

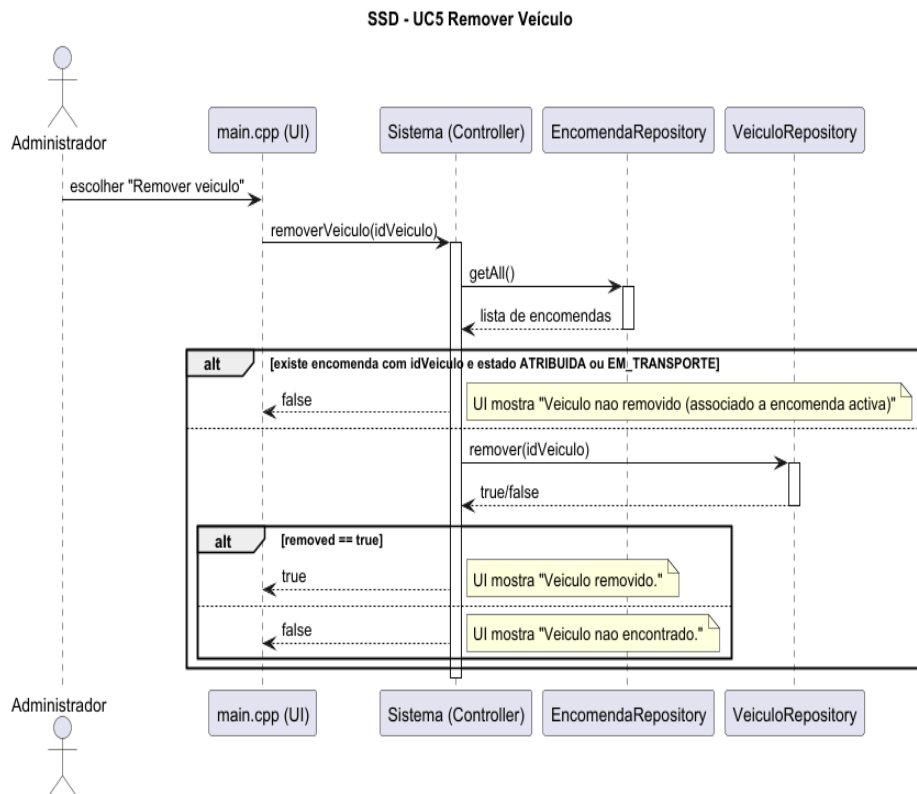


Fig. 7. SSD correspondente ao caso de uso Remover Veículo.

7.6 SSD: UC 6 — Remover Cliente

O System Sequence Diagram do caso de uso “Remover Cliente” (Fig.8) representa o processo de eliminação de um cliente do sistema por parte do administrador. Após a identificação do cliente, o sistema verifica a existência de encomendas associadas que ainda se encontrem ativas. Se tais encomendas existirem, a operação é recusada,

garantindo a consistência dos dados do sistema. Caso contrário, o cliente é removido e o sistema confirma a operação ao administrador.

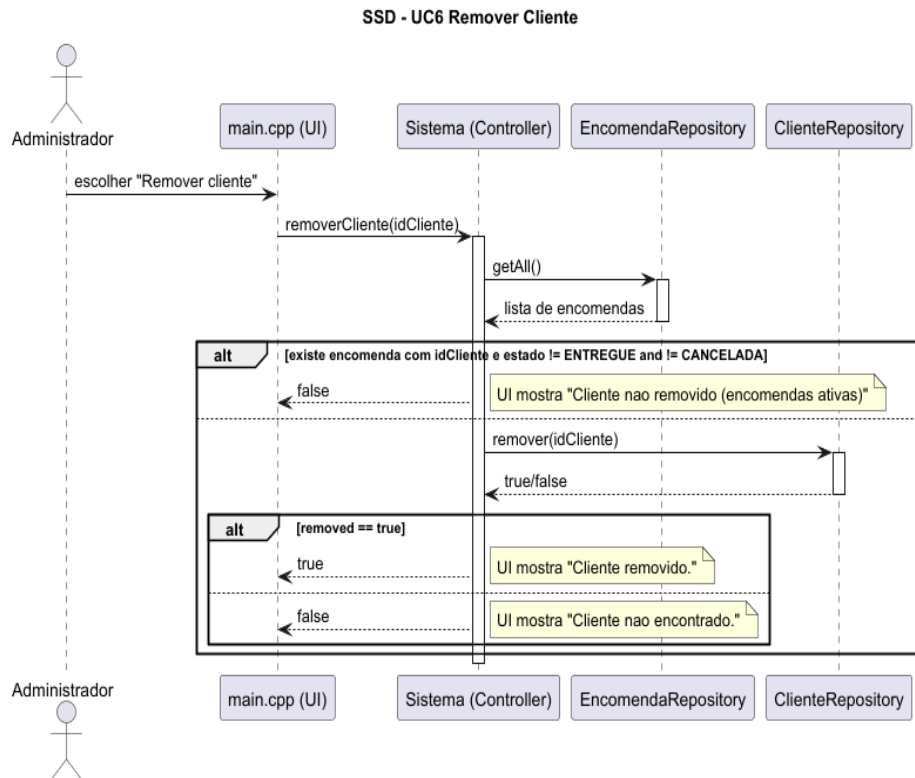


Fig. 8. SSD correspondente ao caso de uso Remover Cliente.

7.7 SSD: UC 7 — Consultar Informação Global

A O System Sequence Diagram do caso de uso “Consultar Informação Global” (Fig. 9) ilustra a interação entre o administrador e o sistema para obtenção de uma visão agregada de todos os dados existentes. Representa a interação entre o administrador e o sistema na consulta de informação global. O sistema responde apresentando os dados relativos a clientes, encomendas, veículos e depósitos, de modo a permitir a supervisão do funcionamento geral da plataforma.

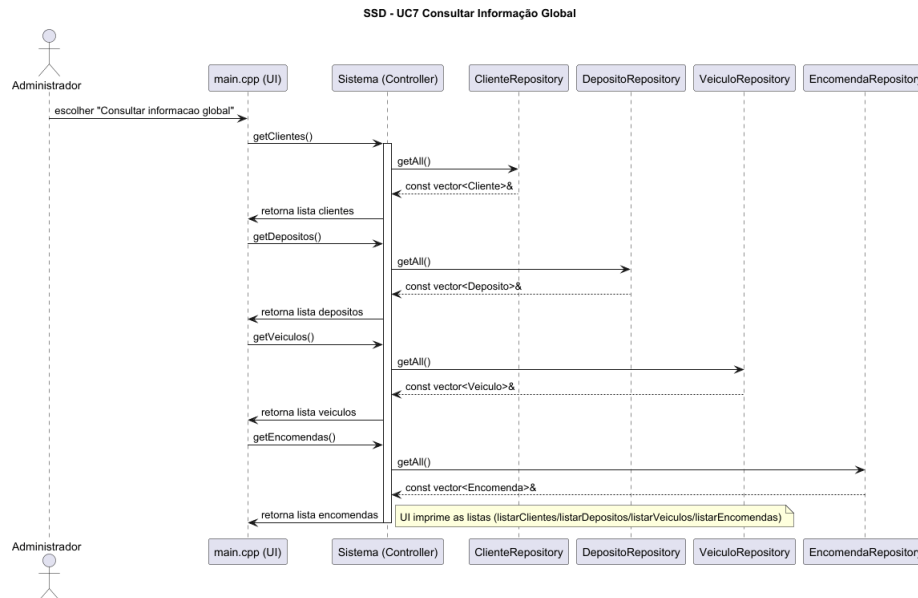


Fig. 9. SSD correspondente ao caso de uso Consultar Informação Global.

8 Desenho do Sistema

Após a análise do domínio e dos casos de uso, passa-se à fase de desenho (design), na qual se define como o sistema será estruturado internamente. Esta estruturação é feita com recurso a vários diagramas UML complementares [4], cada um deles destacando uma perspetiva diferente da solução. Nesta fase aplicam-se igualmente os princípios GRASP (General Responsibility Assignment Software Patterns) [1], um conjunto de padrões que orienta a atribuição de responsabilidades entre classes, promovendo soluções com elevada coesão e baixo acoplamento — duas propriedades fundamentais para que um sistema seja fácil de manter e de evoluir.

8.1 Diagrama de Classes

O diagrama de classes apresenta a estrutura interna do sistema, evidenciando as principais classes, os seus atributos, operações e relações existentes entre elas. Diferentemente do modelo de domínio, que se mantém num plano conceptual, o diagrama de classes já representa decisões de desenho do software e a organização interna da solução.

Como se pode observar na Fig. 10, a classe Sistema atua como controlador principal, centralizando a coordenação das funcionalidades da aplicação. Com a evolução da arquitetura, foram introduzidas classes Repository específicas para cada entidade do domínio — ClienteRepository, EncomendaRepository, VeiculoRepository e DepositoRepository — responsáveis pela gestão e armazenamento dos dados do sistema.

Estas classes passaram a encapsular operações como inserção, remoção, pesquisa, listagem e geração automática de identificadores, utilizando estruturas `std::vector` para armazenar os objetos correspondentes. Desta forma, as entidades Cliente, Encomenda,

Veículo e Deposito mantêm-se focadas apenas na representação dos conceitos e comportamentos do domínio, enquanto os Repositories asseguram a manipulação e organização dos dados.

Esta abordagem permitiu reduzir o acoplamento entre classes, aumentar a coesão do sistema e aplicar de forma mais clara princípios GRASP, especialmente os padrões Controller e Low Coupling, contribuindo para uma solução mais modular, reutilizável e de fácil manutenção.

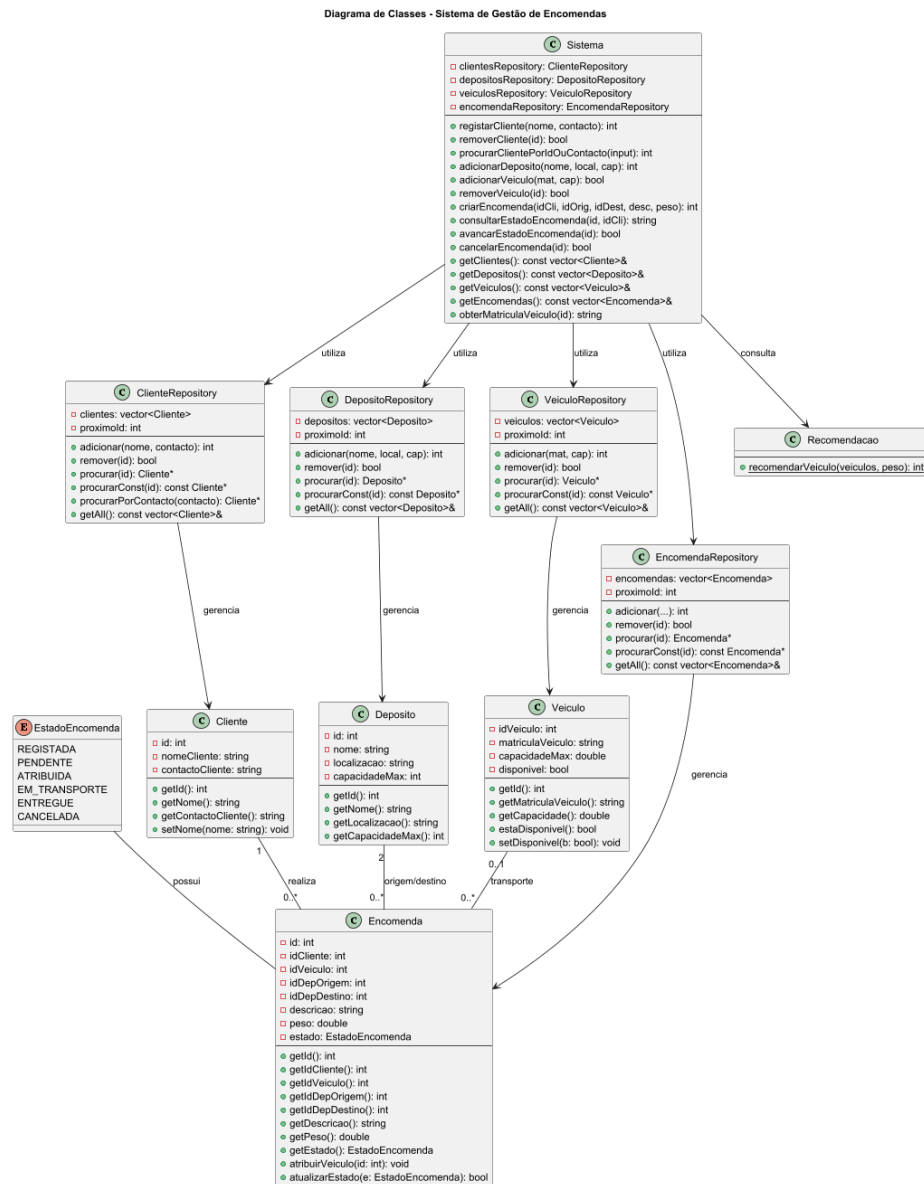


Fig. 10. Diagrama de classes do sistema de gestão de encomendas.

8.2 Diagrama de Sequência

O diagrama de sequência difere do diagrama de sequência de sistema apresentado na secção anterior por mostrar também a colaboração interna entre objetos, e não apenas entre o ator e o sistema como um todo. A Fig. 11 ilustra a interação entre os principais objetos do sistema durante a criação de uma encomenda, evidenciando a colaboração entre o controlador e as classes de domínio: como o pedido do cliente é recebido pelo Sistema, como é criada a Encomenda, como se verifica a disponibilidade do Veículo e como é finalmente confirmada a criação.

Diagrama de Sequência Geral - Sistema de Gestão de Encomendas

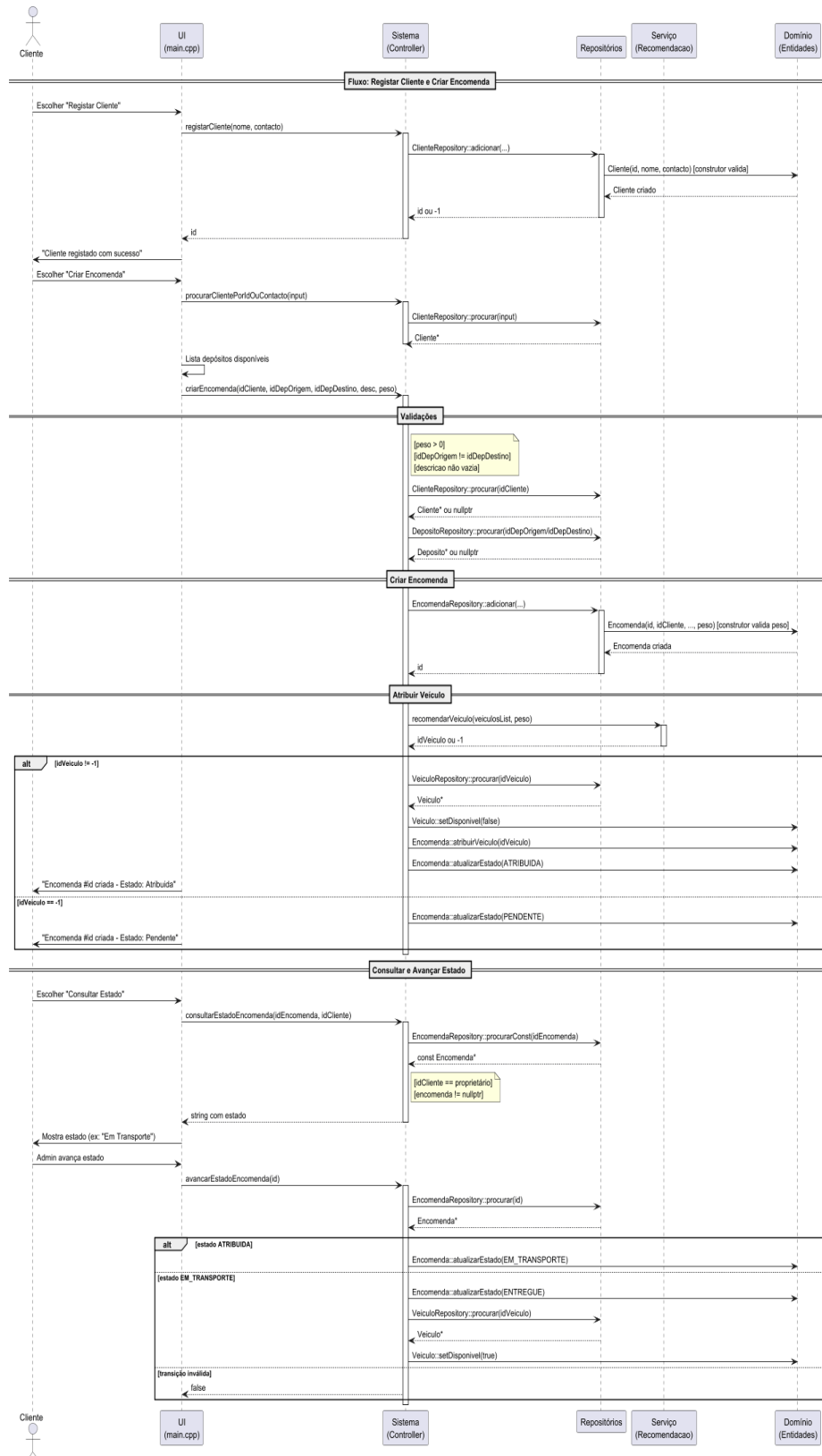


Fig. 11. Diagrama de sequência para a criação de uma encomenda.

8.3 Diagrama de Estados da Encomenda

O atributo estado da Encomenda, apresentado no modelo de domínio (secção 5), é central para o acompanhamento do ciclo de vida de cada pedido. Um diagrama de estados UML permite representar, de forma rigorosa, quais os estados em que uma encomenda pode encontrar-se e quais as transições válidas entre eles, desencadeadas por eventos do sistema ou por ações explícitas dos utilizadores.

A Fig. 12 apresenta o diagrama de estados proposto para a Encomenda. Quando uma encomenda é criada, entra no estado Registada. A partir desse estado, se existir um veículo disponível com capacidade adequada, a encomenda transita para Atribuída; caso contrário, fica no estado Pendente, aguardando que um veículo se torne disponível. Uma vez atribuída, o início efetivo do transporte leva a encomenda ao estado Em Transporte e, quando a entrega é confirmada, ao estado final Entregue. A qualquer momento antes da entrega, o processo pode ser interrompido através da transição Cancelar, conduzindo a encomenda ao estado final Cancelada. Este modelo de estados suporta diretamente o requisito não funcional de fiabilidade (secção 4.2), ao garantir que as transições inválidas — por exemplo, passar diretamente de Registada a Entregue, sem transporte — são rejeitadas pela lógica do sistema.

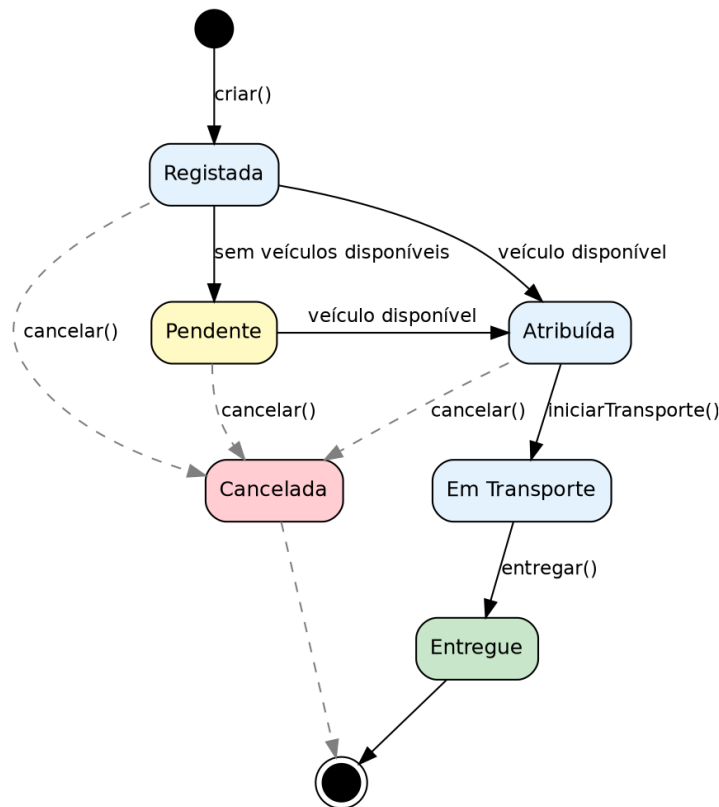


Fig. 12. Diagrama de estados da Encomenda, indicando as transições entre os estados Registrada, Pendente, Atribuída, Em Transporte, Entregue e Cancelada.

8.4 Aplicação de Padrões GRASP

Os padrões GRASP [1] constituem um conjunto de boas práticas para a atribuição de responsabilidades em sistemas orientados a objetos. Não são receitas de código, mas sim princípios de desenho que ajudam a decidir, perante cada responsabilidade identificada, qual a classe mais adequada para a assumir. A Tabela 8 resume os principais padrões GRASP aplicados no desenho do presente sistema, indicando, para cada um, o problema que resolve, a decisão de desenho tomada e a justificação correspondente.

Tabela 9. Padrões GRASP aplicados no desenho do sistema de gestão de encomendas.

Padrão	Classe / Elemento	Justificação	Resultado
Controller	Sistema	É a classe fachada que recebe os pedidos vindos da interface de utilizador e os encaminha para as classes de domínio adequadas.	Mantém a interface de consola desacoplada da lógica do domínio.
Information Expert	Cada classe de domínio	A responsabilidade de responder a cada pergunta é atribuída à classe que possui a informação necessária: Veículo sabe se está disponível (<code>estaDisponivel()</code>), Encomenda sabe como atualizar o seu estado (<code>atualizarEstado()</code>).	Comportamento junto dos dados, evitando classes "anémicas".
Creator	Sistema cria Encomenda	O Sistema agrega as encomendas ao longo da sua existência e dispõe, no momento da criação, de todos os dados necessários (cliente, depósitos, peso, descrição).	Criação centralizada e coerente.
High Cohesion	Todas as classes	Cada classe agrupa apenas responsabilidades fortemente relacionadas entre si: Cliente trata de	Classes mais pequenas, focadas e fáceis de compreender.

		dados do cliente, Veículo da disponibilidade e capacidade, etc.	
Low Coupling	Arquitetura em camadas ui → controller → domain	A interface depende apenas do controlador, e o controlador depende apenas das classes de domínio; não há dependências em sentido inverso.	Alterações num pacote têm impacto reduzido nos restantes.
Pure Fabrication	Classe Sistema	A classe Sistema não corresponde a um conceito do domínio real (não existe "um sistema" no mundo físico); foi fabricada para manter a coesão das classes de domínio e centralizar a coordenação.	Domínio limpo de responsabilidades técnicas.
Protected Variations	Diagrama de estados da Encomenda	A lógica de transições de estado é encapsulada no método atualizarEstado(), protegendo o resto do sistema de futuras alterações no conjunto de estados ou nas regras de transição.	Flexibilidade para evoluir o ciclo de vida das encomendas.

8.5 Diagrama de Pacotes

O diagrama de pacotes (Fig. 12) apresenta a organização do sistema em agrupamentos lógicos de classes, promovendo modularidade, separação de responsabilidades e facilidade de manutenção. Na solução proposta são considerados cinco pacotes principais: um pacote de interface de **utilizador** (ui), responsável pelo menu de interação em consola; um pacote de **controlo** (controller), onde reside a classe Sistema; um pacote **repositório** (repository) para gerar Id, Procurar, Remove e Obter Informação geral; um pacote de **Serviço** (service) que tem parte da lógica do código com recomendar Veículos; e um pacote de **domínio** (domain), que reúne as classes Cliente, Encomenda, Veículo e Depósito. Esta estrutura materializa, a nível arquitetural, o padrão Low Coupling discutido na secção anterior.

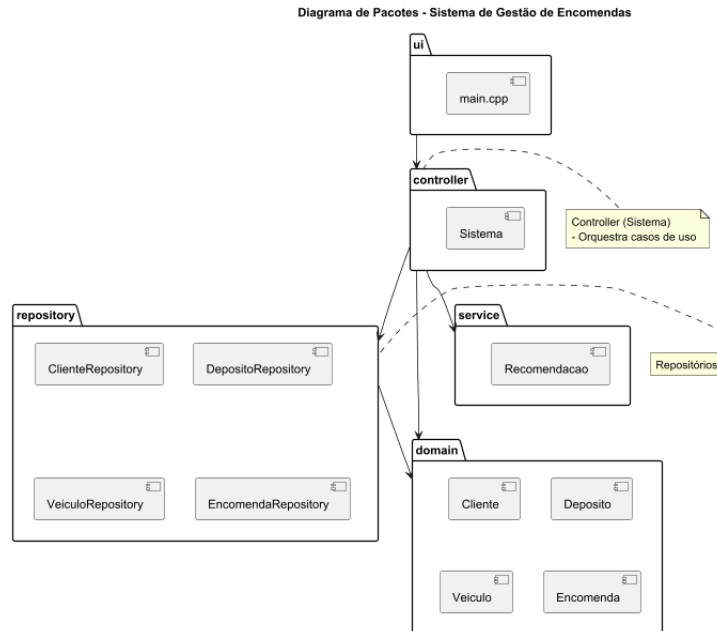


Fig. 12. Diagrama de pacotes do sistema.

8.6 Diagrama de Componentes

O diagrama de componentes (Fig. 13) ilustra a arquitetura do sistema na forma de componentes funcionais que comunicam entre si. Desta perspetiva, evidencia-se a separação entre a interface do utilizador e os módulos centrais de gestão — gestão de encomendas e gestão de veículos — coordenados por um Sistema Central.

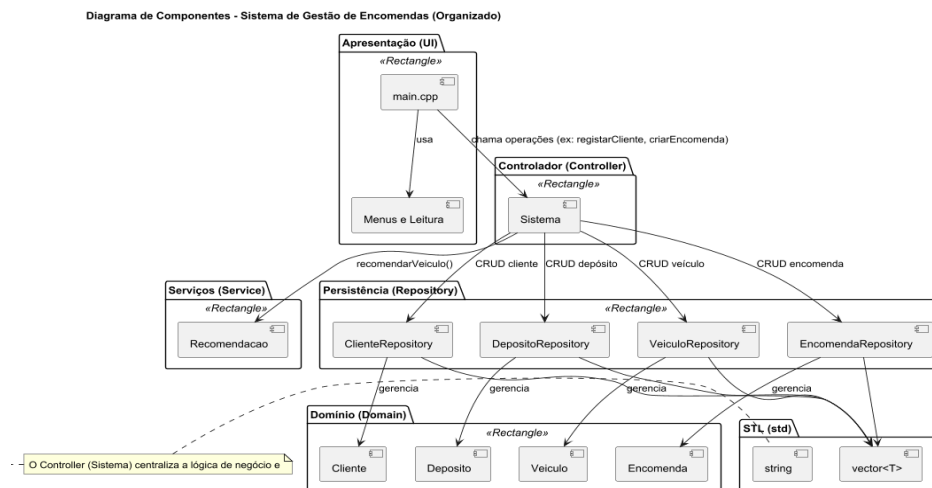


Fig. 13. Diagrama de componentes do sistema.

8.7 Diagrama de Instalação

Finalmente, o diagrama de instalação (deployment diagram) representa a distribuição física dos componentes de software pelos dispositivos de hardware onde serão executados. Uma vez que o sistema é uma aplicação local em C++, executada em modo de consola, o diagrama de instalação é relativamente simples, como se verifica na Fig. 14: a aplicação correrá integralmente no computador do utilizador, sem necessidade de infraestrutura de rede ou de servidores externos.

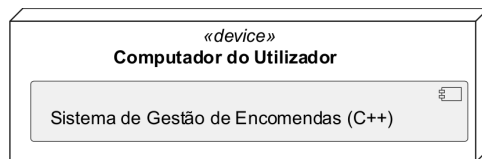


Fig. 14. Diagrama de instalação do sistema.

9 Implementação

9.1 Estrutura do Projeto

A implementação do sistema foi desenvolvida em linguagem C++, seguindo uma organização modular coerente com os princípios definidos na fase de desenho. O projeto encontra-se estruturado em várias camadas, refletindo diretamente o diagrama de pacotes apresentado anteriormente. A estrutura principal do código encontra-se organizada na diretoria *src/*, contendo diferentes subpastas com responsabilidades bem definidas:

- main.cpp*, correspondente à interface de utilizador;
- controller/*, onde se encontra a classe Sistema;
- domain/*, contendo as entidades do domínio;
- repository/*, responsável pela gestão e armazenamento de dados em memória;
- service/*, que inclui lógica adicional, nomeadamente a classe Recomendação.

Esta organização segue uma arquitetura em camadas, promovendo uma separação clara entre interface, lógica de controlo e modelo de domínio, contribuindo para a redução do acoplamento entre componentes.

9.2 Arquitetura do Sistema

A aplicação segue uma arquitetura em camadas com características semelhantes ao padrão Model-View-Controller (MVC). A camada de interface (View) é implementada em *main.cpp*, sendo responsável pela interação com o utilizador. A camada de controlo é representada pela classe Sistema, que atua como intermediária entre a interface e o modelo.

Por fim, a camada de domínio inclui as entidades que representam os elementos fundamentais do sistema.

Adicionalmente, são utilizadas classes do tipo Repository para armazenamento de dados e uma classe de serviço, Recomendacao, responsável pela lógica de seleção de

veículos. Esta arquitetura permite um desenho modular e bem organizado, facilitando a manutenção e evolução do sistema.

9.3 Classes Principais

O sistema é composto por várias classes que refletem diretamente os conceitos identificados no modelo de domínio.

A classe **Cliente** é responsável por representar um utilizador do sistema, armazenando dados como identificador, nome e contacto. A classe **Veiculo** representa os meios de transporte disponíveis, incluindo atributos como matrícula, capacidade e estado de disponibilidade. A classe **Deposito** modela os pontos físicos de recolha e entrega.

A classe **Encomenda** desempenha um papel central, armazenando informações sobre o cliente, os depósitos de origem e destino, o veículo atribuído, o peso e o estado da encomenda. Para além disso, contém lógica que controla as transições de estado, garantindo a coerência do seu ciclo de vida.

A classe Sistema atua como controlador principal, coordenando os diferentes casos de uso, efetuando validações e comunicando com os repositórios e serviços. Por sua vez, a classe **Recomendacao** encapsula a lógica de seleção de veículos, permitindo identificar o veículo mais adequado consoante a capacidade e disponibilidade.

Os **repositórios** (*Repository*) são responsáveis por armazenar os dados em memória, utilizando estruturas baseadas em *std::vector*, e disponibilizam operações de inserção, remoção e pesquisa.

9.4 Fluxo Principal do Sistema

O funcionamento do sistema pode ser observado através do fluxo de criação de uma encomenda. Inicialmente, o utilizador introduz os dados necessários na interface. Estes dados são enviados para a classe **Sistema**, que valida condições como a existência do cliente, a diferença entre depósitos de origem e destino e o peso da encomenda.

Se as validações forem satisfeitas, a encomenda é criada no repositório correspondente. De seguida, o sistema utiliza a classe **Recomendacao** para identificar um veículo disponível com capacidade adequada. Caso exista um veículo adequado, este é associado à encomenda e marcado como indisponível. Caso contrário, a encomenda é registada com estado pendente.

Este fluxo demonstra a interação entre interface, controlador, domínio e serviços, refletindo diretamente o diagrama de sequência definido na fase de desenho.

9.5 Aplicação dos Padrões GRASP

Na implementação foram aplicados vários padrões GRASP, garantindo uma atribuição adequada de responsabilidades.

O padrão *Controller* é implementado pela classe Sistema, que recebe os pedidos da interface e coordena as operações. O padrão *Information Expert* é visível nas classes de domínio, uma vez que cada entidade detém a informação necessária para executar as suas

operações. O padrão *Creator* é também aplicado à classe Sistema, responsável pela criação das encomendas.

Adicionalmente, foram seguidos os princípios de *High Cohesion* e *Low Coupling*, garantindo que cada classe possui responsabilidades bem definidas e que as dependências entre classes são minimizadas.

9.6 Estruturas de Dados e Validações

O sistema utiliza estruturas da Standard Template Library (STL), nomeadamente `std::vector`, para armazenar coleções de entidades, e `std::string` para tratamento de texto. Esta abordagem simplifica a gestão de memória e permite uma implementação eficiente. Foram também implementadas validações em diferentes níveis do sistema. Ao nível das classes de domínio, são verificadas condições como valores inválidos ou campos vazios, sendo lançadas exceções do tipo `std::invalid_argument`. Ao nível do controlador, são validadas regras de negócio, como a existência de entidades ou a consistência dos dados introduzidos.

10 Testes e Validação

10.1 Estratégia de Teste

A validação do sistema foi realizada principalmente através de testes manuais, utilizando a interface em consola implementada em `main.cpp`. Estes testes permitiram verificar o comportamento do sistema em diferentes cenários de utilização.

Apesar da ausência de uma suíte automatizada de testes, foram definidos vários cenários que garantem a verificação das funcionalidades essenciais do sistema.

10.2 Testes do Módulo Cliente

Foram realizados testes relacionados com o registo e validação de clientes. Entre os cenários testados destacam-se o registo de um cliente com dados válidos, a tentativa de registo com dados inválidos e a tentativa de registo com informação duplicada.

10.3 Testes de Encomendas

Os testes ao módulo de encomendas incluíram a criação de encomendas com depósitos válidos e inválidos, a verificação do comportamento do sistema quando não existem veículos disponíveis e a validação das transições de estado. Foram também testadas operações como o cancelamento de encomendas e a consulta do seu estado por parte do cliente, garantindo que apenas o utilizador associado pode aceder às suas encomendas.

10.4 Testes de Administração

No contexto das funcionalidades administrativas, foram realizados testes para validação da adição e remoção de veículos, bem como para a consulta de informação global do sistema. Foram ainda considerados cenários de erro, como a tentativa de remover um

veículo inexistente ou em utilização, assegurando que o sistema responde de forma consistente.

10.5 Limitações e Trabalhos Futuros

Atualmente, o sistema ainda não possui testes automatizados unitários, sendo essa uma melhoria relevante para futuras versões. A introdução de testes unitários permitiria validar de forma sistemática o comportamento das classes de domínio, dos repositórios e do controlador, aumentando a robustez da aplicação.

11 Glossário

A Tabela 10 reúne os termos técnicos utilizados ao longo deste relatório, acompanhados das respectivas definições, de modo a facilitar a leitura do documento por parte de leitores com formações distintas.

Tabela 10. Glossário de termos técnicos utilizados no relatório.

Administrador	Ator do sistema responsável pela gestão e supervisão global, incluindo a administração de veículos, a remoção de contas e a consulta de informações agregadas sobre clientes, encomendas, veículos e depósitos.
Cliente	Utilizador do sistema que cria encomendas e acompanha o respetivo estado ao longo do processo de entrega.
Depósito	Ponto físico de recolha ou entrega de encomendas previsto no sistema. O projeto contempla seis depósitos distribuídos por uma pequena cidade.
Encomenda	Pedido de transporte criado por um cliente, caracterizado por uma descrição, um peso, um depósito de origem, um depósito de destino e um estado atual.
Estado da Encomenda	Situação em que uma encomenda se encontra no seu ciclo de entrega. Os estados previstos são Registada, Pendente, Atribuída, Em Transporte, Entregue e Cancelada (ver secção 8.3).
FURPS+	Modelo de classificação de requisitos não funcionais que agrupa a qualidade de um sistema em cinco categorias — Functionality, Usability, Reliability, Performance e Supportability — com um grupo adicional "+" para restrições técnicas, legais e outras.
GRASP	Sigla de General Responsibility Assignment Software Patterns. Conjunto de padrões que orientam a atribuição de

	responsabilidades em sistemas orientados a objetos, promovendo elevada coesão e baixo acoplamento.
Sistema	Aplicação responsável pela gestão das encomendas, veículos e depósitos, e que centraliza a coordenação das funcionalidades oferecidas ao Cliente e ao Administrador.
SSD	System Sequence Diagram, ou Diagrama de Sequência de Sistema. Diagrama UML que descreve a sequência temporal de mensagens trocadas entre um ator externo e o sistema visto como uma caixa preta.
UML	Unified Modeling Language. Linguagem gráfica padronizada utilizada para modelar e documentar sistemas de software orientados a objetos.
Use Case	Descrição de uma funcionalidade do sistema na perspetiva de um ator externo, incluindo pré-condições, pós-condições, fluxo principal, fluxos alternativos e regras de negócio.
User Story	Descrição curta e informal de uma funcionalidade, habitualmente no formato "Como [utilizador], quero [ação], para que [benefício]".
Low Coupling	Princípio de design que recomenda reduzir as dependências entre classes, facilitando a manutenção e evolução do sistema.
High Cohesion	Princípio que indica que cada classe deve ter uma responsabilidade bem definida e focada numa única tarefa.
Recomendacao	Componente responsável pela seleção do veículo mais adequado para uma encomenda, com base na disponibilidade e capacidade.
Repository	Componente responsável pelo armazenamento e gestão de dados em memória, incluindo operações de inserção, remoção e pesquisa.
STL (Standard Template Library)	Classe principal da aplicação que atua como controlador, coordenando as operações e implementando os casos de uso.

12 Conclusão

O trabalho desenvolvido no âmbito deste relatório técnico permitiu estruturar, de forma coerente, a proposta de um sistema de gestão de encomendas orientado a objetos para uma pequena cidade com seis pontos

de distribuição. Ao longo do projeto foram percorridas as principais etapas da análise e concepção de software, incluindo a identificação de requisitos funcionais e não funcionais, a modelação do domínio, a especificação de casos de uso e a elaboração de diferentes diagramas UML que descrevem o funcionamento e a estrutura interna da aplicação.

Um dos principais desafios encontrados ao longo deste processo esteve relacionado com a definição da arquitetura interna do sistema, nomeadamente a organização das responsabilidades entre as classes Cliente, Encomenda, Veículo e Depósito, bem como a comunicação entre a interface, o controlador e o domínio. Com a evolução do projeto, foram introduzidas classes Repository específicas para cada entidade, permitindo separar a lógica de armazenamento da lógica do domínio e tornando o sistema mais modular, organizado e de mais fácil manutenção. Esta abordagem contribuiu igualmente para uma aplicação mais clara dos princípios GRASP, especialmente no que respeita aos padrões Controller, Low Coupling e High Cohesion.

A utilização de notação UML e de princípios de Engenharia de Software revelou-se fundamental para garantir uma estrutura coerente e consistente ao longo do desenvolvimento do trabalho. A matriz de rastreabilidade permitiu ainda assegurar que todas as user stories identificadas possuem correspondência nos respetivos casos de uso, contribuindo para a completude da análise realizada.

Apesar de o sistema ainda se encontrar em desenvolvimento, a fase atual permitiu estabelecer uma base sólida para a futura implementação em C++, definindo previamente os requisitos, os comportamentos esperados e a estrutura geral da aplicação. O trabalho realizado contribuiu também para consolidar conhecimentos relacionados com Programação Orientada a Objetos, arquitetura em camadas, modelação de sistemas e organização de projetos de software.

Como evolução futura, prevê-se a conclusão da implementação do sistema, a introdução de persistência de dados em ficheiros ou bases de dados, a realização de testes automatizados e a integração de funcionalidades adicionais, como notificações ao cliente, cálculo automático de custos de transporte e interfaces gráficas mais avançadas, permitindo tornar o sistema mais completo e mais próximo de um contexto real de utilização.

13 Referências

1. Larman, C.: Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. 3rd edn. Prentice Hall, Upper Saddle River (2005)
2. Sommerville, I.: Software Engineering. 10th edn. Pearson Education, Harlow (2016)
3. Pressman, R.S., Maxim, B.R.: Software Engineering: A Practitioner's Approach. 8th edn. McGraw-Hill Education, New York (2014)
4. Object Management Group: Unified Modeling Language (UML) Specification, version 2.5.1. OMG Document formal/2017-12-05 (2017), <https://www.omg.org/spec/UML>
5. IEEE: IEEE Recommended Practice for Software Requirements Specifications. IEEE Std 830-1998. IEEE, New York (1998)
6. Sousa, P.B., Freitas, C.: Fundamentos de Desenvolvimento de Software — Material da Unidade Curricular. Instituto Superior de Engenharia do Porto (2025)
7. Grady, R.B.: Practical Software Metrics for Project Management and Process Improvement. Prentice Hall, Englewood Cliffs (1992)